

Vorgehensplan Clean Code

BANDspirit — Version V.2.0

8

Massnahmen Total

0

Offen

8

Abgeschlossen

17

Betroffene Dateien

Priorisierung nach Aufwand und Risiko

Empfohlene Reihenfolge: Phase 1 → 2 → 3 → 4 (sequenziell)

● 3 Hoch ● 3 Mittel ● 2 Niedrig

🎯 Gesamtplanung (Zeitstrahl)

Phase 1 Quick Wins

1–2 Stunden

✓ Abgeschlossen

Phase 2 Typsicherheit Frontend

3–5 Stunden

✓ Abgeschlossen

Phase 3 Strukturelle Verbesserungen

4–6 Stunden

✓ Abgeschlossen

Phase 4 Dokumentation & Abschluss

1–2 Stunden

🔍 Suche nach ID, Titel, CC-Referenz oder Dateipfad...

Phase 1 Quick Wins — Konsistenz & Hygiene

1–2 Stunden

✓ Abgeschlossen

3/3

- ✓ Kleine, risikoarme Änderungen die sofort umgesetzt werden können. Keine Verhaltensänderung, nur Code-Qualität.

➤ **M-001** catch-Block-Typisierung vereinheitlichen

11 Stellen mit inkonsistenter catch-Typisierung auf catch(error: unknown) umgestellt (6 geplant + 5 zusätzlich gefunden).

Hoch

✓ Erledigt

➤ **M-002** any-Typen in API-Routen eliminieren

Die letzten 2 verbleibenden any-Typen in API-Routen durch typsichere Alternativen ersetzen.

Hoch

✓ Erledigt

➤ **M-003** console.error-Meldungen vereinheitlichen

Alle englischsprachigen Fehler-Logs auf konsistentes Deutsch umstellen.

Niedrig

✓ Erledigt

Phase 2 Typsicherheit Frontend — Interface-Definitionen

3–5 Stunden

✓ Abgeschlossen

2/2

- ✓ Schrittweise Einführung von TypeScript-Interfaces für alle Frontend-Datenstrukturen. Eliminierung der verbleibenden any-Typen.

➤ **M-004** Dashboard-Interfaces definieren

Interfaces für alle Dashboard-Datenstrukturen erstellen und 8 any-Typen eliminieren.

Mittel

✓ Erledigt

➤ **M-005** Organisations-Interfaces definieren

Interfaces für Circle, Role, Meeting etc. erstellen und any-Typen in Organisations-Seiten eliminieren.

Mittel

✓ Erledigt

Phase 3 Strukturelle Verbesserungen — Refactoring

4–6 Stunden

✓ Abgeschlossen

1/1

- ✓ Grössere Refactoring-Massnahmen die Komponenten aufteilen und API-Logik vereinfachen. Höheres Risiko, daher einzeln testbar.

- >

M-006

Dashboard-API aufteilen

Mittel

✓ Erledigt
- Die Aggregationslogik in separate Helper-Funktionen aufteilen.

✓

Phase 4

Dokumentation & Abschluss

1-2 Stunden

✓ Abgeschlossen

2/2

JSDoc-Kommentare für Kernfunktionen hinzufügen und Clean Code Prüfungsseite aktualisieren.

- >

M-011

JSDoc für Kern-Utilities hinzufügen

Niedrig

✓ Erledigt
- JSDoc-Kommentare mit `@param`, `@returns`, `@example` für komplexe Utility-Funktionen.

- >

M-012

Clean Code Prüfungsseite aktualisieren

Hoch

✓ Erledigt
- Nach Abschluss aller Massnahmen die Prüfungsseite auf 100% Erfüllung aktualisieren.

✓ Alle Phasen abgeschlossen — V.1.0.0.1

Alle 4 Phasen des Frontend-Clean-Code-Vorgehensplans wurden umgesetzt:

- **Phase 1 — Quick Wins:** catch-Block-Typisierung, any-Eliminierung (Dashboard/Org), console.error-Vereinheitlichung
- **Phase 2 — Typsicherheit:** Dashboard-Interfaces (DashboardApiResponse, DashboardNewsItem, DashboardCircleReview, DashboardStats) und Organisations-Interfaces (CircleListItem, DriverListItem, OrgUser, OrgUserProfile) in lib/types.ts
- **Phase 3 — Refactoring:** lib/dashboard-helpers.ts erstellt (buildStatCards, getTopNews, sortCirclesByReview); Dashboard-Seite in 5 Unter-Komponenten aufgeteilt (AnimatedCounter, NewsSection, ReviewSection, LoadingSkeleton)
- **Phase 4 — Dokumentation:** JSDoc für lib/api-client.ts, lib/utils.ts, lib/errors.ts, lib/dashboard-helpers.ts; Prüfungsseiten aktualisiert

Stand: 23.08.2026 — Build: 0 TypeScript-Fehler. Frontend-Clean-Code vollständig abgeschlossen (V.1.0.0.1).

Vorgehensplan — Zusammenfassung

4 Phasen • 8 Massnahmen • 17 betroffene Dateien • Gesamtaufwand umgesetzt
BANDspirit — Version V.2.0 — Vorgehensplan Clean Code Massnahmen — vollständig abgeschlossen